

DEVOPS CASE STUDY REPORT

Con Cung Company

Candidate: Võ Công Huân

GitHub: <https://github.com/conghuan99/devops-casestudy>

May 2026

Tasks Completed

Task 0: Kubernetes Deployment | Task 1: Jenkins CI/CD | Task 2: Infrastructure as Code | Task 3: Monitoring & Observability

Contents

1. Project Overview	3
1.1 Environment	3
1.2 Repository Structure	3
2. Pipeline Design	4
2.1 Architecture	4
2.2 Pipeline Stages	4
2.3 Failure Handling & Rollback	4
3. How to Run the Demo	4
3.1 Prerequisites	4
3.2 Scenario A: Quick App Deployment & Monitoring (Fast Track)	4
3.3 Scenario B: Full Enterprise CI/CD Pipeline Setup (Advanced Track)	5
3.4 Access Points	6
3.5 Trigger Pipeline	6
4. Task 0 – Kubernetes Deployment	8
4.1 Resources Created	8
4.2 Why kubectl instead of Helm	8
4.3 Rollback Strategy	8
4.4 Scaling	8
4.5 Common K8s Issues	8
5. Task 2 – Infrastructure as Code (Ansible)	9
5.1 Approach	9
5.2 Playbook: setup-devops-env.yml	9
5.3 Reproducibility & Config Drift Prevention	9
5.4 How to Run	9
6. Task 3 – Monitoring & Observability	10
6.1 Stack	10
6.2 Key Metrics	10
6.3 Alert Conditions	10
6.4 Debugging a Production Issue	10
7. Troubleshooting	12
Issue 1 – Loki WAL Permission Denied	12
Issue 2 – Docker Compose --force-recreate Race Condition	12

1. Project Overview

This report covers the DevOps platform implementation for Con Cung, demonstrating CI/CD pipeline, Kubernetes deployment, IaC provisioning, and full-stack monitoring.

1.1 Environment

Component	Detail
Cloud	AWS EC2 t3.medium, Singapore region
OS	Ubuntu 24.04 LTS
Kubernetes	k3s (lightweight Kubernetes)
CI/CD	Jenkins Controller + Agent via Docker Compose
Application	Python Flask API with prometheus-flask-exporter
Monitoring	Prometheus + Loki + Promtail + Grafana

1.2 Repository Structure

```
devops-casestudy/
├── docker-compose.yml      # Jenkins + Monitoring stack
├── Jenkinsfile             # Multi-stage pipeline definition
├── jenkins-agent.Dockerfile # Custom agent: Python3, Docker CLI, kubectl
├── ansible/
│   └── setup-devops-env.yml # Ansible playbook (Task 2)
├── flask_app/
│   ├── app.py              # Flask app with /metrics endpoint
│   ├── test_app.py         # Unit tests
│   ├── requirements.txt
│   └── Dockerfile
├── k8s/
│   ├── deployment.yaml     # 2 replicas, rolling update strategy
│   └── service.yaml        # NodePort 30080
├── monitoring/
│   ├── prometheus.yml      # Scrape: Flask, Jenkins
│   ├── loki-config.yml     # WAL + compactor config
│   ├── promtail-config.yml # Pod log collection
│   └── grafana/
│       ├── provisioning/
│       │   ├── datasources/datasources.yml
│       │   └── dashboards/dashboards.yml
│       └── dashboards/
│           ├── flask-dashboard.json
│           └── jenkins-dashboard.json
```

2. Pipeline Design

2.1 Architecture

Jenkins Controller + Agent via Docker Compose. Controller handles scheduling/UI; Agent executes all build steps (Python, Docker, kubectl). GitHub webhook triggers pipeline automatically on push to main.

2.2 Pipeline Stages

#	Stage	Description
1	Checkout	Clone source from GitHub main branch
2	Test	pytest unit tests — pipeline fails immediately if any test fails
3	Docker Build	Build image tagged flask-app:\${BUILD_NUMBER} for immutable versioning
4	Push	Push to Docker Hub using credentials stored in Jenkins Credentials Store
5	Deploy	kubectl set image → rollout status (60s timeout) → auto-rollback on fail

2.3 Failure Handling & Rollback

- Credentials (Docker Hub) stored in Jenkins Credentials Store, injected via withCredentials — never hardcoded.
- Deploy stage: if rollout status times out after 60s, kubectl rollout undo is triggered automatically.
- post { failure } block captures logs and marks build as failed on any stage error.
- Image versioning via BUILD_NUMBER (e.g., flask-app:42) allows rollback to any specific previous build.

3. How to Run the Demo

This section provides a step-by-step guide to run and evaluate the system. To facilitate the evaluation process, the demonstration is split into two independent scenarios based on the evaluator's specific focus.

3.1 Prerequisites

Ensure your target machine (e.g., AWS EC2 Ubuntu instance) has the following baseline configurations:

- Operating System: Ubuntu 22.04 or 24.04 LTS.
- Inbound Security Group Rules: Open the following TCP ports on AWS Console:
 - 30080: Flask Application (Kubernetes NodePort)
 - 8080: Jenkins Controller UI
 - 3000: Grafana Dashboard
 - 9090: Prometheus Server UI

3.2 Scenario A: Quick App Deployment & Monitoring (Fast Track)

Use Case: Select this scenario if you only want to quickly spin up the Flask Application on Kubernetes (k3s) alongside the Monitoring Stack, without configuring the entire Git-triggered Jenkins pipeline.

Step 1: Install Dependencies

Ensure Docker, Docker Compose, and k3s are active. If using a fresh server, you can trigger our automated Ansible playbook locally to handle all software installations in one click:

```
sudo apt update && sudo apt install software-properties-common -y
sudo add-apt-repository --yes --update ppa:ansible/ansible && sudo apt install ansible -y
ansible-playbook -i "localhost," -c local ansible/setup-devops-env.yml
```

Step 2: Clone the Project

```
git clone https://github.com/conghuan99/devops-casestudy.git
cd devops-casestudy
```

Step 3: Run Monitoring Stack via Docker Compose

```
docker-compose up -d prometheus grafana loki promtail
```

Step 4: Deploy Flask App Directly to k3s Cluster

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

Step 5: Verification

- Flask Application UI: Access via `http://<YOUR_SERVER_IP>:30080/health`
- App Live Metrics: Access via `http://<YOUR_SERVER_IP>:30080/metrics`

3.3 Scenario B: Full Enterprise CI/CD Pipeline Setup (Advanced Track)

Use Case: Select this scenario to replicate the complete end-to-end DevOps automation process, testing the multi-stage Jenkinsfile, parallel testing, automated Docker image build/push, and progressive rolling updates with automatic rollbacks.

Step 1: Repository Replication (Mandatory)

Because the CI/CD pipeline requires read/write access to trigger automated steps, you cannot use the read-only source repository directly.

1. Fork or import this repository into your personal GitHub account: `https://github.com/<your-username>/devops-casestudy`
2. Update the environment variables block at the top of the Jenkinsfile to map your personal Docker Hub namespace:

```
environment {
    DOCKER_IMAGE = "YOUR_PERSONAL_DOCKERHUB_USERNAME/flask_app"
    DOCKER_TAG   = "v${BUILD_NUMBER}"
}
```

3. Commit and push this change back to your personal GitHub repository.

Step 2: Spin Up Jenkins Infrastructure

Execute the standard compose file to host both the Jenkins Controller and its decoupled, pre-configured SSH Build Agent:

```
docker-compose up -d jenkins-controller jenkins-agent
```

Step 3: Unlock Jenkins & Install Dependencies

1. Access the master dashboard at `http://<YOUR_SERVER_IP>:8080`.
2. Extract the auto-generated initial password to unlock the setup wizard:

```
docker logs jenkins-controller
```
3. Click "Install suggested plugins" and create your primary admin account.
4. Install Prometheus Metrics Plugin: Go to Manage Jenkins -> Plugins -> Available Plugins, search for "Prometheus Metrics", and install it. This opens the pipeline metric endpoint at `http://<YOUR_SERVER_IP>:8080/prometheus/`.

Step 4: Configure Secure Jenkins Credentials

To prevent credential leaking, the pipeline abstracts sensitive tokens. You must register the following credential inside the Jenkins store:

- Navigate to Manage Jenkins -> Credentials -> System -> Global credentials -> Add Credentials.
- Kind: Username with password
- ID: dockerhub-credentials (Must exactly match this string)
- Username: Your Docker Hub username.
- Password: Your Docker Hub Personal Access Token (PAT).

Step 5: Create and Fire the Pipeline Job

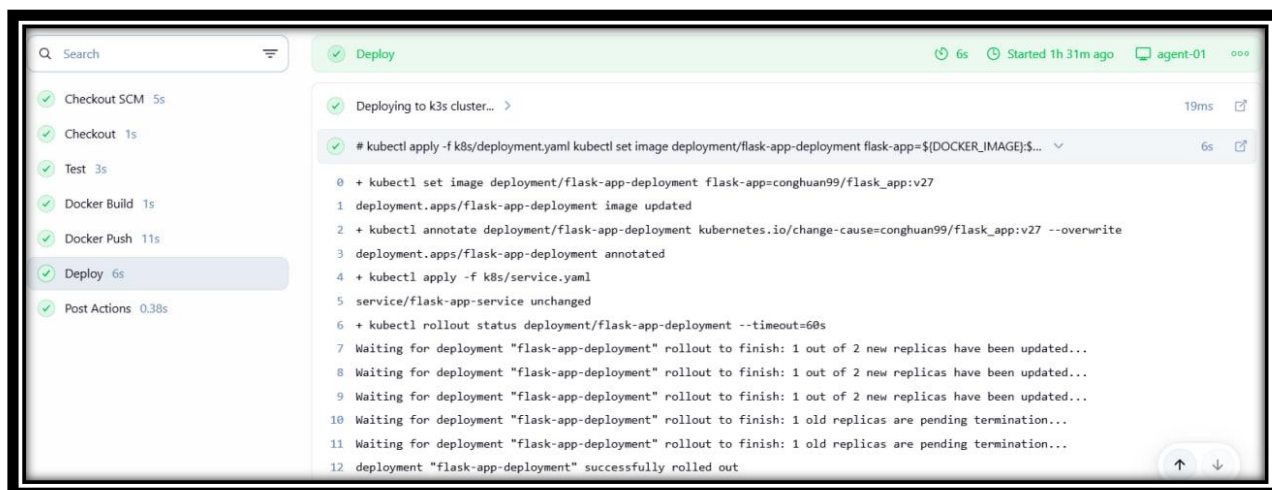
1. On the Jenkins main dashboard, click New Item, name it "flask-app-pipeline", select Pipeline, and click OK.
2. Scroll down to the Pipeline configuration section:
 - Definition: Select "Pipeline script from SCM".
 - SCM: Select "Git".
 - Repository URL: Paste your duplicated personal GitHub repo URL.
 - Branch Specifier: */main.
 - Script Path: Jenkinsfile.
3. Click Save, then click "Build Now" to watch the full pipeline execute from source checkout down to the final Kubernetes rolling deployment.

3.4 Access Points

Service	URL	Credentials
Jenkins	<code>http://<IP>:8080</code>	admin / (initial setup password)
Grafana	<code>http://<IP>:3000</code>	admin / admin123
Prometheus	<code>http://<IP>:9090</code>	No auth
Flask App	<code>http://<IP>:30080</code>	No auth

3.5 Trigger Pipeline

1. Push any commit to main branch on GitHub, OR go to Jenkins UI → flask_app_pipeline → Build Now.
2. Watch stages progress in Jenkins UI.



3. Verify: curl <http://<IP>:30080/health>
4. Check Grafana Flask dashboard — request metrics appear within 30s of deployment.

4. Task 0 – Kubernetes Deployment

4.1 Resources Created

- Deployment (k8s/deployment.yaml): 2 replicas, RollingUpdate (maxSurge:1, maxUnavailable:0), resource limits.
- Service (k8s/service.yaml): NodePort 30080 — exposes app for traffic and Prometheus scraping.
- Image tag: flask-app:\${BUILD_NUMBER} — each build produces a unique, immutable tag.

4.2 Why kubectl instead of Helm

kubectl set image was used because this is a single-service demo app. Helm adds overhead (chart structure, values files) only justified for multi-service or multi-environment deployments. In production with dev/staging/prod environments, Helm would be the correct choice.

4.3 Rollback Strategy

Automatic rollback in the Deploy stage:

```
kubectl rollout status deployment/flask-app-deployment --timeout=60s || kubectl  
rollout undo deployment/flask-app-deployment
```

Kubernetes retains previous ReplicaSets (revisionHistoryLimit:3), so rollback re-points the Deployment to the previous ReplicaSet instantly.

```
ubuntu@ip-172-31-26-141:~/test_cc/devops-casestudy$ kubectl rollout history deployment/flask-app-deployment  
deployment.apps/flask-app-deployment  
REVISION  CHANGE-CAUSE  
6         <none>  
7         kubectl set image deployment/flask-app-deployment flask-app=conghuan99/flask_app:v18 --record=true  
12        kubectl set image deployment/flask-app-deployment flask-app=conghuan99/flask_app:v18 --record=true  
13        conghuan99/flask_app:v20  
14        conghuan99/flask_app:v21  
16        conghuan99/flask_app:  
17        conghuan99/flask_app:v22  
18        conghuan99/flask_app:v24  
19        conghuan99/flask_app:v25  
20        conghuan99/flask_app:v26  
21        conghuan99/flask_app:v27
```

4.4 Scaling

- Manual: kubectl scale deployment flask-app --replicas=N
- Autoscaling: HorizontalPodAutoscaler (HPA) targeting CPU > 70%. Node-level scaling requires adding EC2 instances to the k3s cluster.

4.5 Common K8s Issues

Issue	Root Cause	Resolution
ImagePullBackOff	Wrong registry URL or missing credentials	Verify docker login and image tag exists
Pod stuck Pending	Insufficient CPU/Memory on node	Reduce resource requests or free capacity
CrashLoopBackOff	Application error on startup	kubectl logs <pod> to inspect stdout

5. Task 2 – Infrastructure as Code (Ansible)

5.1 Approach

Ansible was chosen over Terraform because the target is a single existing EC2 instance (configuration management, not resource provisioning). Ansible is agentless, SSH-based, requires no state file.

5.2 Playbook: setup-devops-env.yml

Task Group	What it does
System Prerequisites	apt update, install curl/git/python3/gnupg/ca-certificates
Docker CE	Add Docker GPG key + repo, install docker-ce, start service, add ubuntu to docker group
Docker Compose	Download v1.29.2 binary to /usr/local/bin, set executable
k3s	Run official install script, wait for kubeconfig, copy to ~/.kube/config
kubectl	Download matching binary — skips if already exists (idempotent)

5.3 Reproducibility & Config Drift Prevention

- All versions declared as vars (docker_compose_version, k3s_version, kubectl_version) — one variable change re-provisions with a different version.
- Idempotent: re-running the playbook multiple times produces the same result (binary existence checks, state: present).
- Config drift prevention: playbook can be scheduled (e.g., via Ansible AWX) to detect and correct drift automatically.
- Multi-environment: separate inventory files (inventory-dev.ini, inventory-prod.ini) with environment variable files (group_vars/).

5.4 How to Run

```
sudo apt update

sudo apt install software-properties-common -y

sudo add-apt-repository --yes --update ppa:ansible/ansible

sudo apt install ansible -y

ansible-playbook -i ansible/inventory.ini ansible/setup-devops-env.yml
```

6. Task 3 – Monitoring & Observability

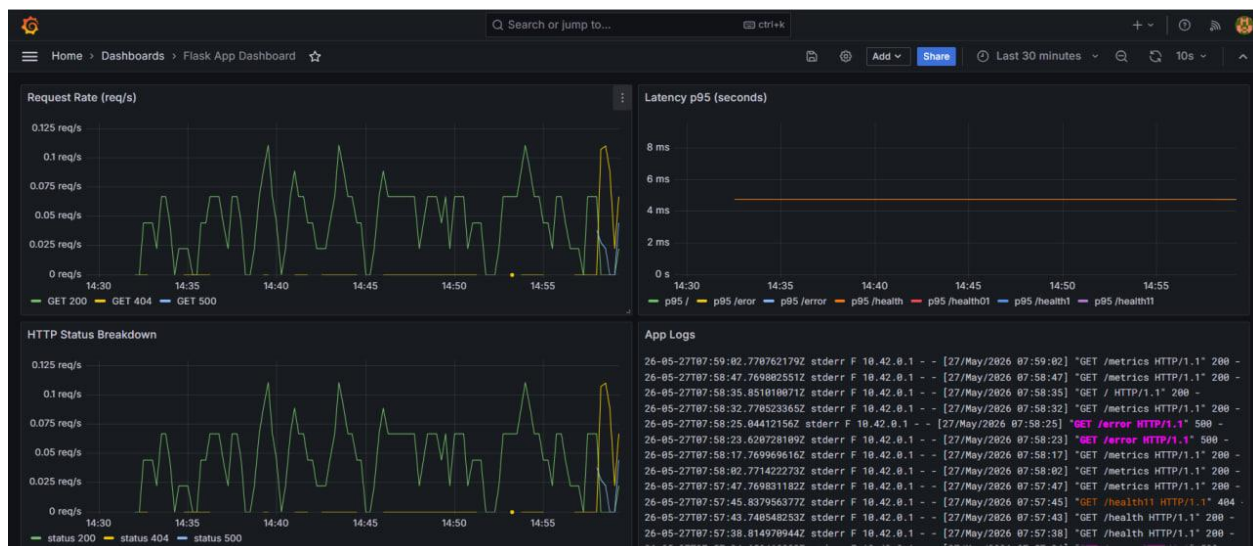
6.1 Stack

Component	Port	Role
Prometheus	9090	Scrapes Flask (:30080/metrics), Jenkins (:8080/prometheus) every 15s
Loki	3100	Log aggregation — receives structured logs from Promtail
Promtail	—	Reads /var/log/pods/*, parses pod/namespace/container labels, ships to Loki
Grafana	3000	2 dashboards: Flask, Jenkins. Datasources + alerts provisioned via files.

6.2 Key Metrics

Application: flask_http_request_total (count by status), flask_http_request_duration_seconds (latency → p95).

Jenkins: default_jenkins_builds_last_build_result (1=success/0=fail), builds_duration_milliseconds_summary, executors_queue_length.



6.3 Alert Conditions

Alert	Condition
Flask 5xx Error Rate High	rate(flask_http_request_total{status=~'5..'}[1m]) > 0.01 for 2m — validated as Firing during testing

The figure shows the Grafana alerting interface. It displays two alerts that are currently firing. The first alert is 'Flask 4xx Error Rate High' and the second is 'Flask 5xx Error Rate High'. Both alerts are in a 'Firing' state and have a 'Summary' field that says 'Flask app có {{ \$values.C }} errors/s'.

State	Name	Health	Summary	Next evaluation	Actions
Firing	Flask 4xx Error Rate High	ok	Flask app có {{ \$values.C }} errors/s	in a few seconds	More
Firing	Flask 5xx Error Rate High	ok	Flask app có {{ \$values.C }} errors/s	in a few seconds	More

6.4 Debugging a Production Issue

1. Check Grafana Flask dashboard — spike in 5xx rate or p95 latency increase pinpoints the time window.

2. Correlate with App Logs panel (Loki) — filter by pod/namespace to find stack traces or error messages.
3. Check System dashboard — high CPU or memory exhaustion can cause latency spikes.
4. Check Jenkins dashboard — last_build_result and build timestamp confirm if issue started post-deployment.
5. Rollback if needed: kubectl rollout undo deployment/flask-app, redeploy after fix.

7. Troubleshooting

Issue 1 – Loki WAL Permission Denied

Symptom	Loki container exited immediately: mkdir /loki/wal: permission denied
Root Cause	Loki runs as non-root uid 10001. Docker volume mount created directory as root:root.
Fix	mkdir -p /data/loki && chown 10001:10001 /data/loki. Added explicit wal and compactor paths in loki-config.yml.

Issue 2 – Docker Compose --force-recreate Race Condition

Symptom	Containers stuck in removing state; new containers failed with port already in use.
Root Cause	Docker Compose v1.29.2 has a known race condition with --force-recreate when containers share a network.
Fix	Replaced with: docker stop <name> && docker rm -f <name> && docker-compose up -d <name>